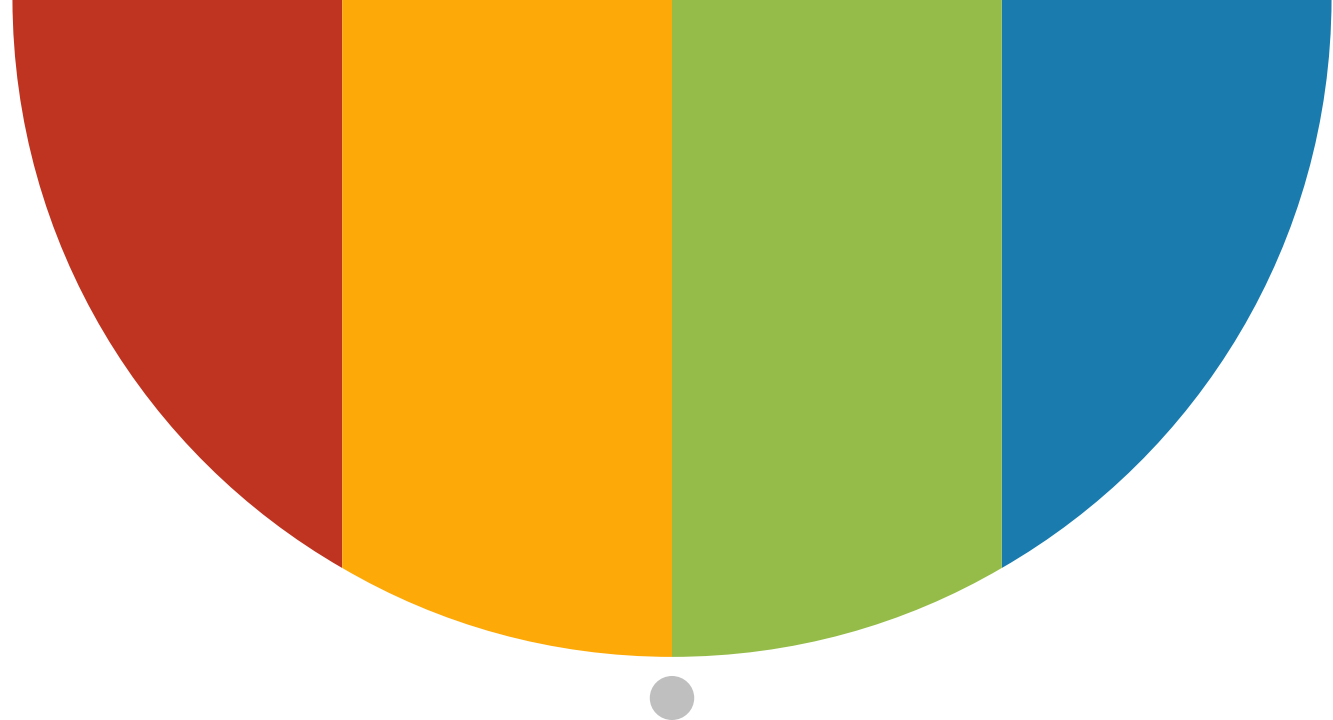




深度学习 Deep Learning

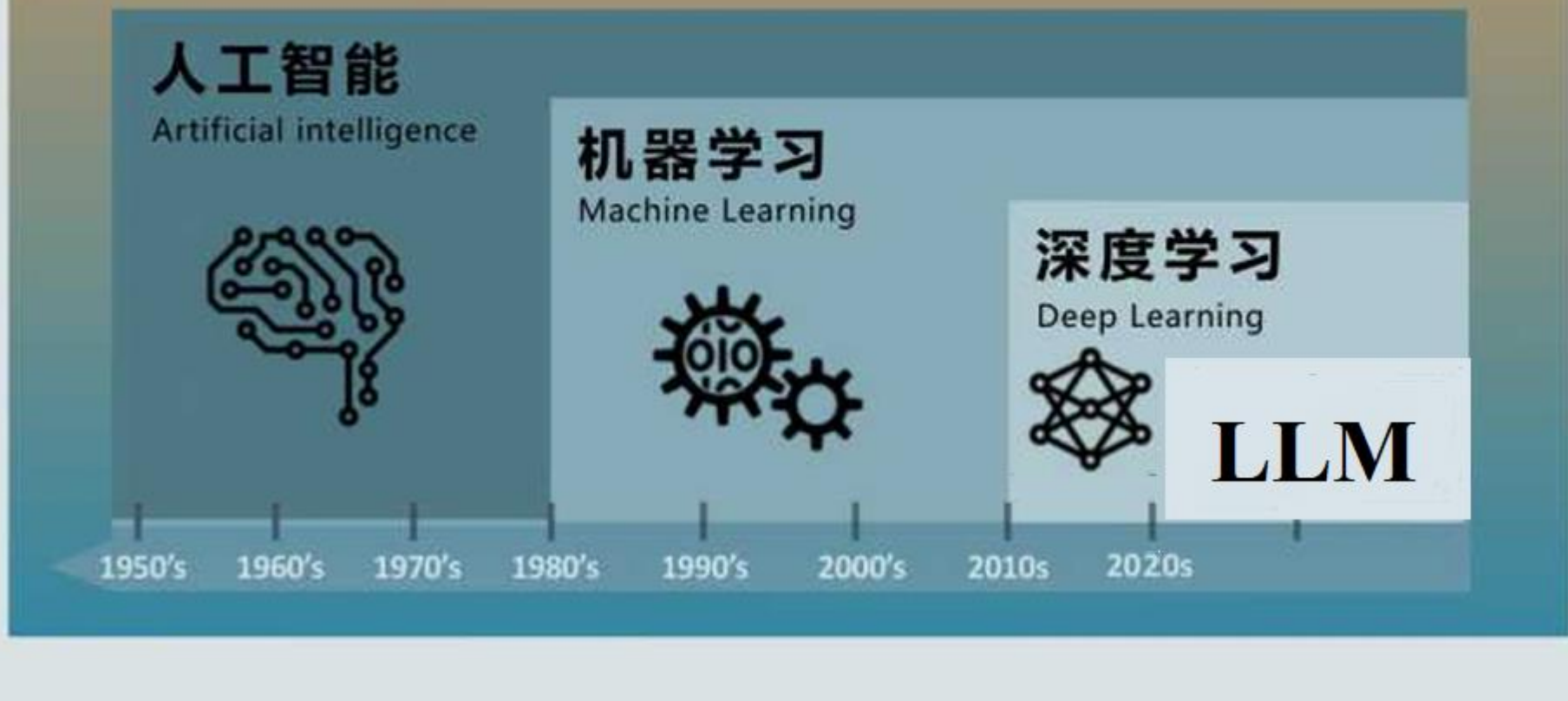
南开大学
孙羽菲



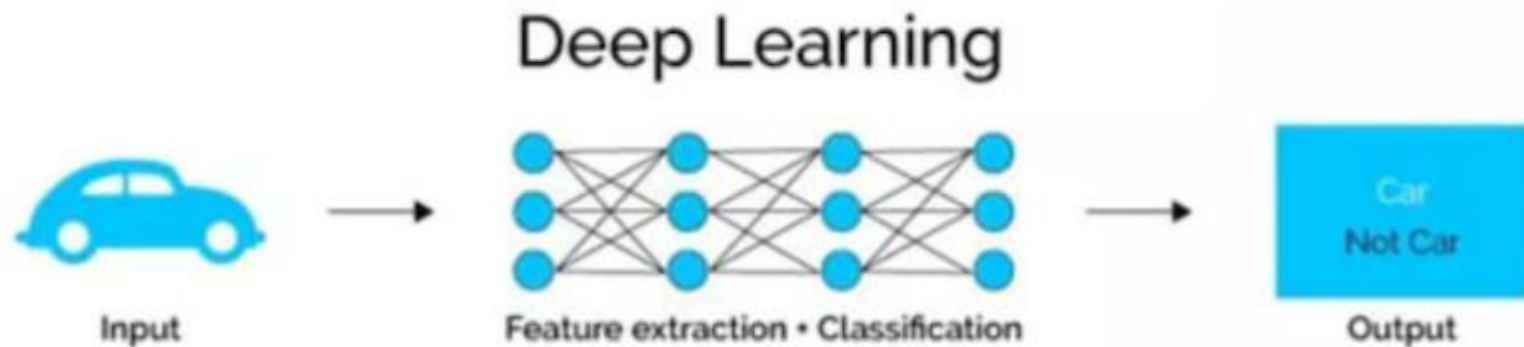
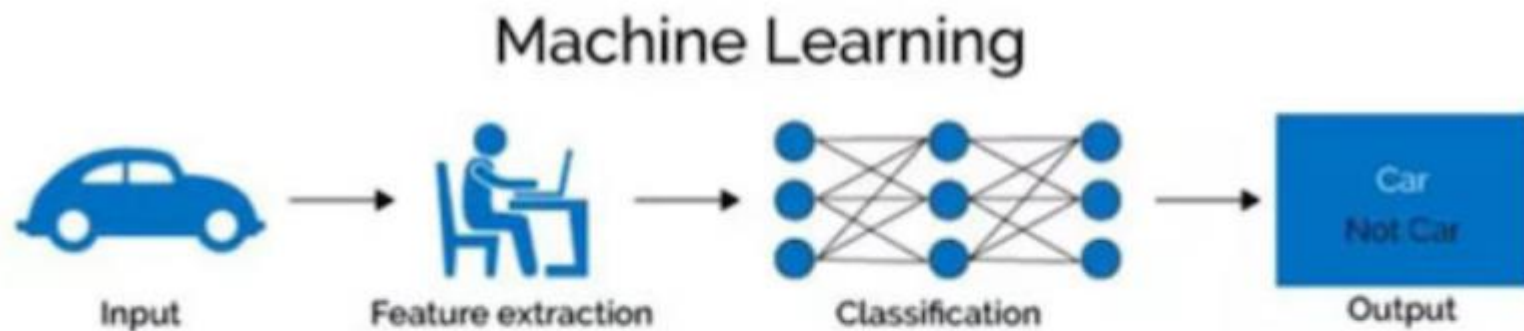
什么是深度学习？

深度学习

人工智能指的是在某种程度上显示出类似人类智能的设备。人工智能有许多技术，但其中一个子集是机器学习——让算法从数据中学习。最后，深度学习是机器学习的一个子集，使用多层神经网络来解决最困难的计算机问题。



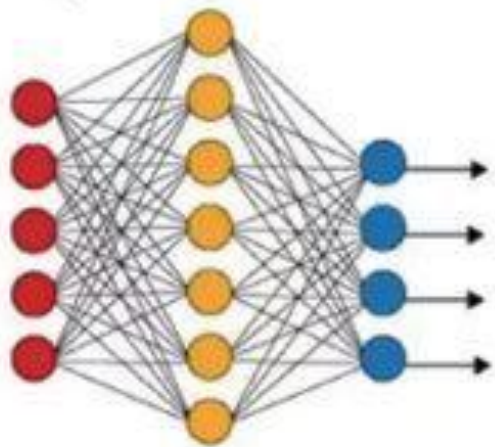
深度学习与机器学习



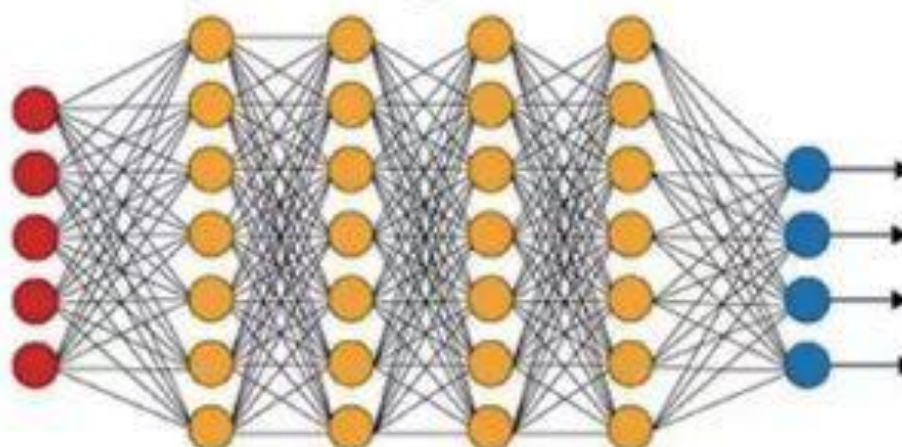
深度学习模型

深度学习利用深度的神经网络模仿人脑的机制来解释数据，含多个隐藏层的多层感知器就是一种典型的深度学习模型网络

Simple Neural Network



Deep Learning Neural Network



● Input Layer

● Hidden Layer

● Output Layer

深度学习发展史-1

● 1940s-1950s 早期神经网络概念

1943年，Warren McCulloch和Walter Pitts发表论文“A logical calculus of the ideas immanent in nervous activity”（神经活动中内在思想的逻辑演算），建立了神经网络和数学模型

● 1950s-1960s：感知机和早期模型

1958年，计算机科学家Frank Rosenblatt提出了两层神经元组成的神经网络，称之为感知器(Perceptrons)

● 1980s-1990s：多层感知机和反向传播

1985年，Geoffrey Hinton等人发表论文A learning algorithm for boltzmann machines，提出了受限玻尔兹曼机 (RBM)。

1986年，Geoffrey Hinton 发明了适用于多层感知器（MLP）的BP（Backpropagation）算法，并采用Sigmoid进行非线性映射，有效解决了非线性分类和训练的问题。

1989年，Yann LeCun等人发表论文Backpropagation Applied to Handwritten Zip Code Recognition（反向传播应用于手写邮政编码识别），使用BP算法训练卷积神经网络（CNN）用于手写数字识别。

深度学习发展史-2

● 1990s-2000：深度学习的形成期

1990年，Jeffrey Elman发表论文Finding structure in time提出 SRNs（也叫 Elman Networks），其核心概念就是今天所熟知的循环神经网络（RNN）。

1991年，Sepp Hochreiter在毕业论文中阐述了梯度消失问题。

1993年，Geoffrey Hinton发表论文Autoencoders, minimum description length and Helmholtz free energy，发表了关于自编码器（Autoencoders）的研究。

1997年，Sepp Hochreiter和Jürgen Schmidhuber发表了论文Long Short-Term Memory，为了解决RNN的梯度消失问题，提出了LSTM。

1998年，Yann LeCun等人发表论文Gradient-based learning applied to document recognition，改进了之前的CNN，提出了LeNet-5。

深度学习发展史-3

● 2000s：深度学习的快速发展

2006年，Geoffrey Hinton等人发表论文A Fast Learning Algorithm for Deep Belief Nets，提出深度信念网络（DBN）。这篇论文被认为是近代的深度学习方法的开始。

2006年，Geoffrey Hinton等人发表论文Reducing the Dimensionality of Data with Neural Networks，提出深度自编码器。

2006年，NVIDIA推出CUDA编程模型（生态），利用GPU的并行处理能力，将GPU用作通用并行计算设备，以加速各种计算任务，而不仅限于图形处理。CUDA大大提升了深度学习算法的效率。

深度学习发展史-4

● 2010s:深度学习的突破与普及

2012年，Alex Krizhevsky、Ilya Sutskever和Geoffrey Hinton的AlexNet在ImageNet 大规模视觉识别挑战赛 (ILSVRC)中取得了巨大成功。

2013年，Google DeepMind提出了Deep Q-Network (DQN)，将深度学习与强化学习相结合。

2013年，Kingma和Welling提出了变分自编码器（VAE）。

2014年，Ian Goodfellow等人提出生成对抗网络（GAN）。

2014年，Google DeepMind提出注意力机制（Attention Mechanism），开创了将注意力机制应用于深度学习模型的先河。

2015年，何凯明等人提出了ResNet（残差网络）。

2015年-2016年，Google，Facebook相继推出TensorFlow、PyTorch 和 Keras。

2016年：Google DeepMind开发的AlphaGo击败了围棋世界冠军李世石，展示了深度强化学习的潜力。

2017年，Google Brain发表了Attention is All You Need，提出了Transformer。

2018年，OpenAI 发表了Improving Language Understanding by Generative Pre-Training，提出了GPT。

2018年，Google AI Language发表了BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding。

深度学习发展史-5

● 2020s:深度学习的繁荣发展

2020年10月22日，Google团队提出了Vision Transformer（ViT）。

2021年2月26日，OpenAI发表了Learning Transferable Visual Models From Natural Language Supervision，提出了CLIP，实现了多模态学习。

2021年7月，Google DeepMind发布了AlphaFold 2.

2022年12月20日，Stability AI发表论文High-Resolution Image Synthesis with Latent Diffusion Models，发布了完全开源的Stable Diffusion.

2022年11月30日，OpenAI发布了GPT-3.5，其产品ChatGPT瞬间成为全球爆品。引起了全球学术界和工业界的大语言模型热潮。

2023年，Albert Gu和Tri Dao提出了Mamba，一种新的不同于Transformer的处理长序列数据的神经网络架构.

2024年2月，OpenAI发布了SORA，实现了较高质量的文本到视频生成。

2024年5月8日，DeepMind提出了AlphaFold 3。

2025年1月15日，深度求索推出DeepSeek，成为全球增长最快的AI工具。

深度学习三要素

数据

庞大的标注数据集
更容易的加工和存储

IMAGENET

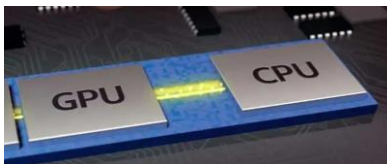


WIKIPEDIA
The Free Encyclopedia



算力

强劲的GPU
分布式并行



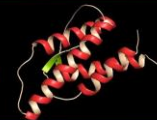
算法

先进的算法
新模型
框架

PyTorch



DeepMind AlphaFold
Crushes Competition



The Artificial Neural Network
News Network

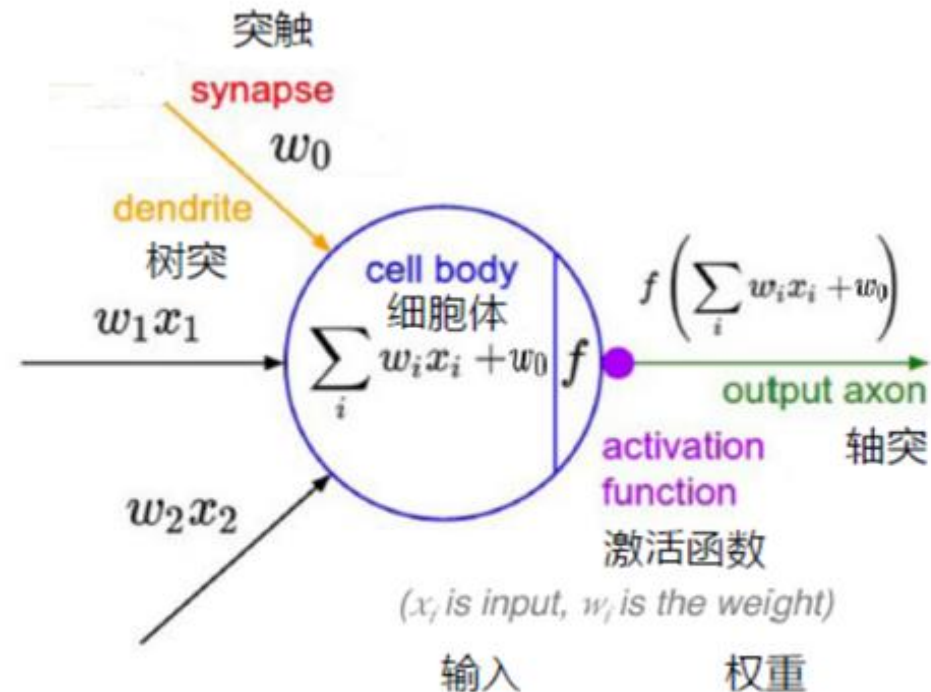
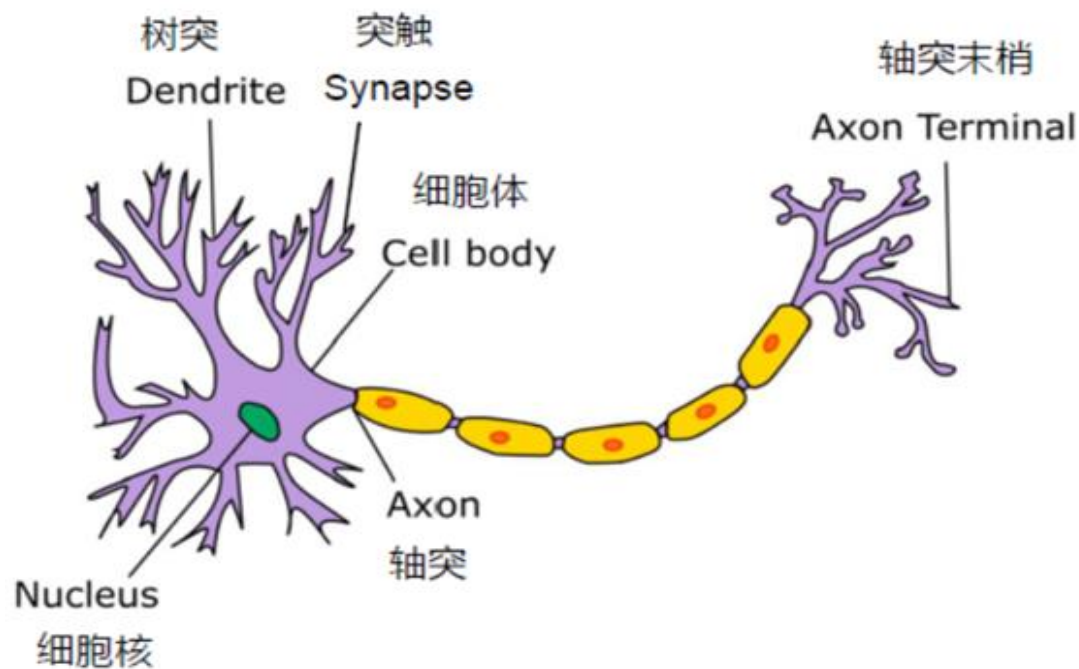


2

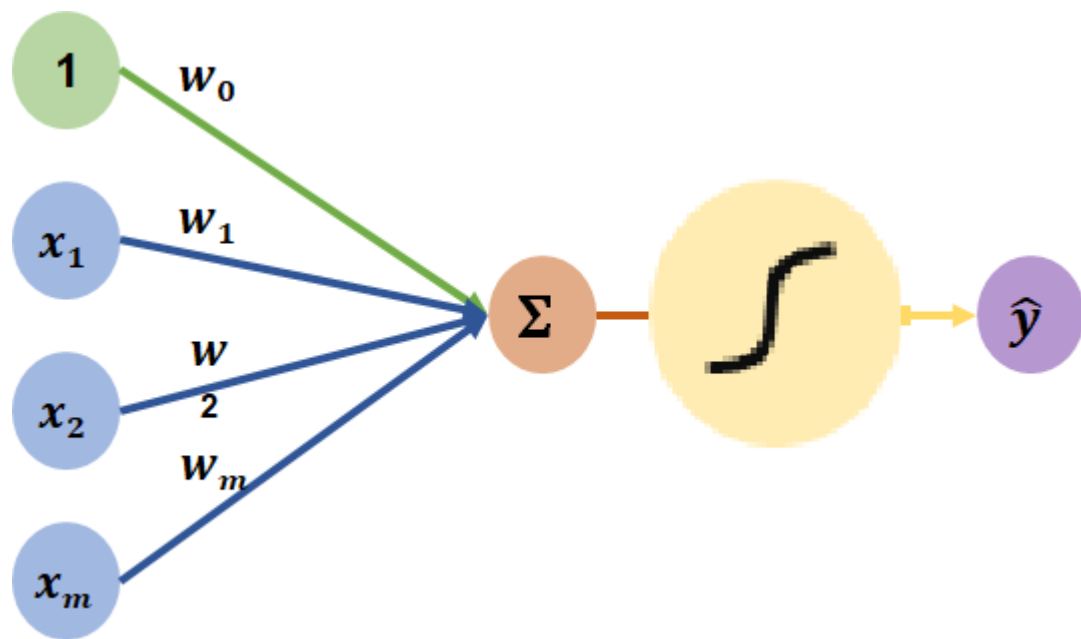
基础知识和概念

感知器、激活函数、损失函数、梯度下降

神经元与感知器 (Perceptron)



感知器 (Perceptron)



Output

$$\hat{y} = f \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

input

activation function

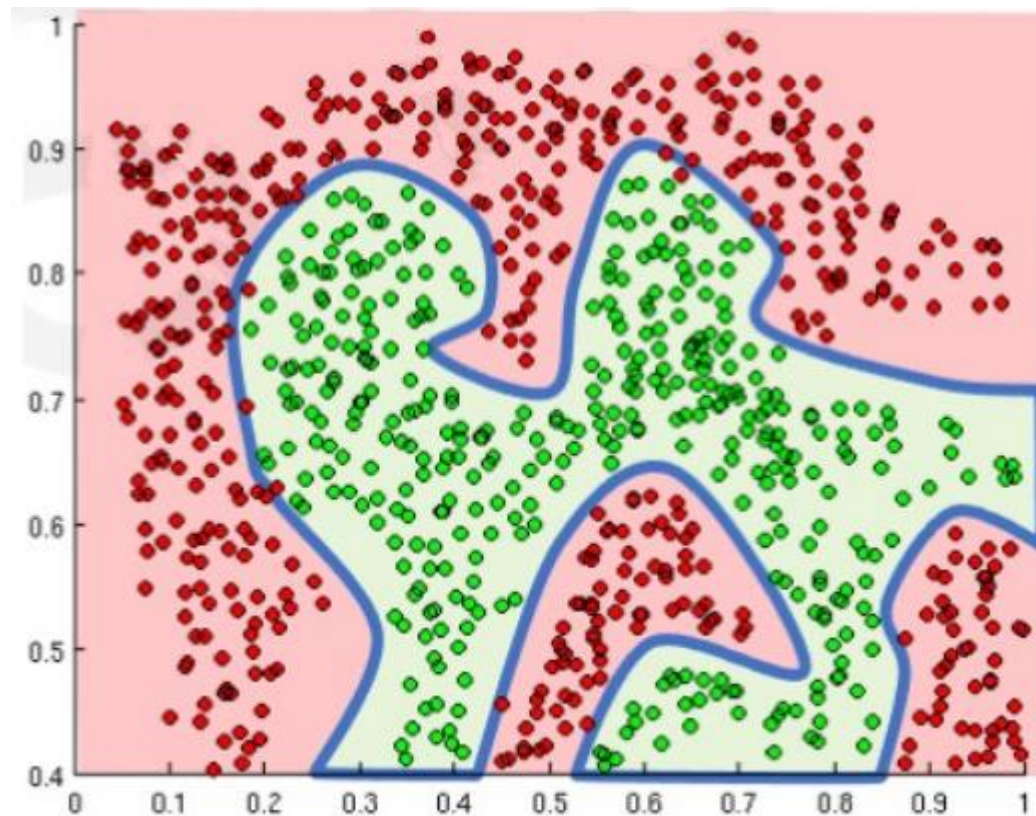
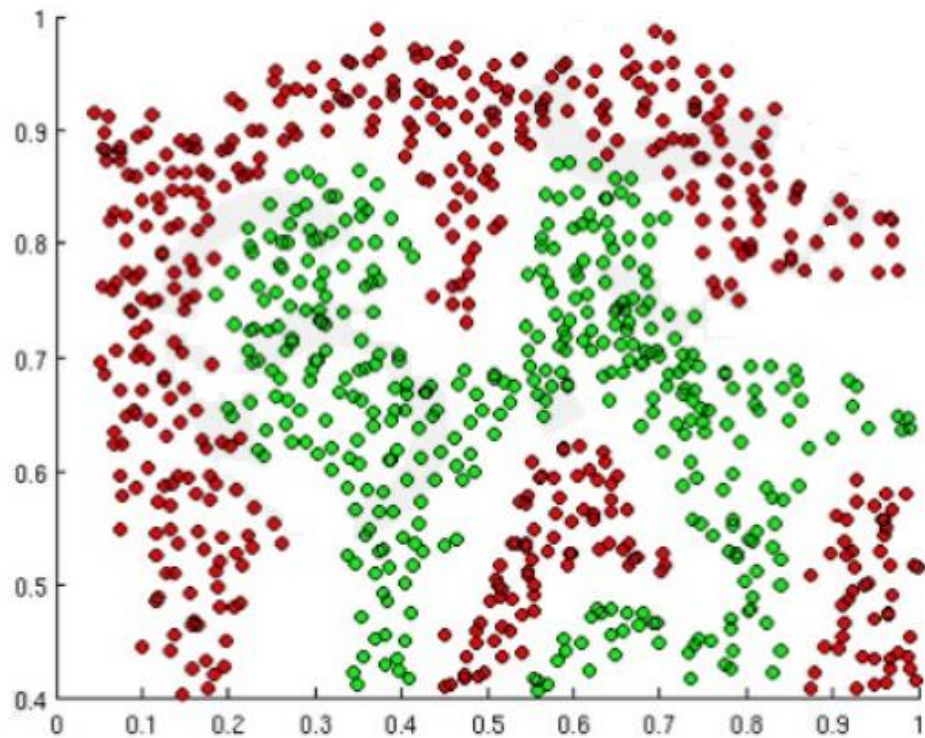
Bias

weights

$$\hat{y} = f(w_0 + X^T W)$$

where: $X = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$ and $W = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$

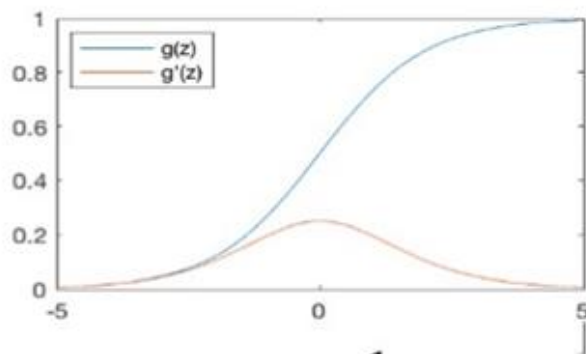
激活函数



激活函数为神经网络引入了非线性因素，使得神经网络可以任意逼近非线性函数

常见激活函数

Sigmoid Function



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

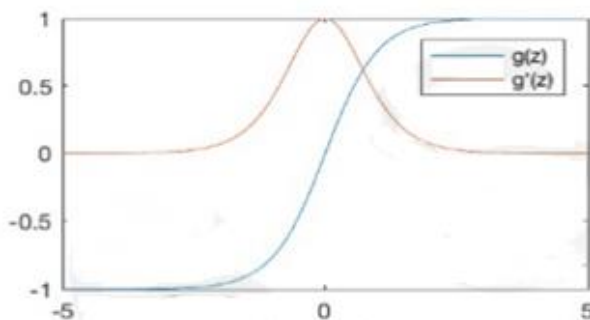
优点:

1. 易于求导。
2. 输出映射在(0,1)之间, 输出标准化。

缺点:

1. 软饱和, 容易造成梯度消失问题。
2. 输出值不是Zero-center, 可能导致模型收敛速度慢。

Hyperbolic Tangent



$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

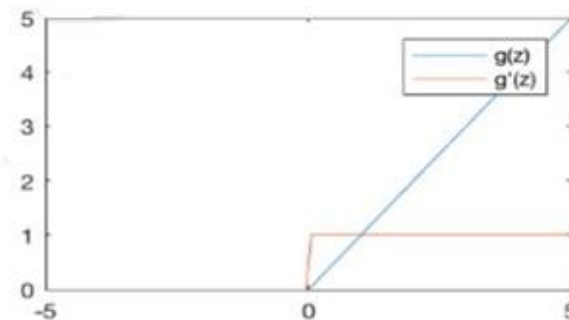
优点:

1. 输出Zero-center。
2. 收敛速度比Sigmoid函数快

缺点:

1. 仍有软饱和现象。

Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

优点:

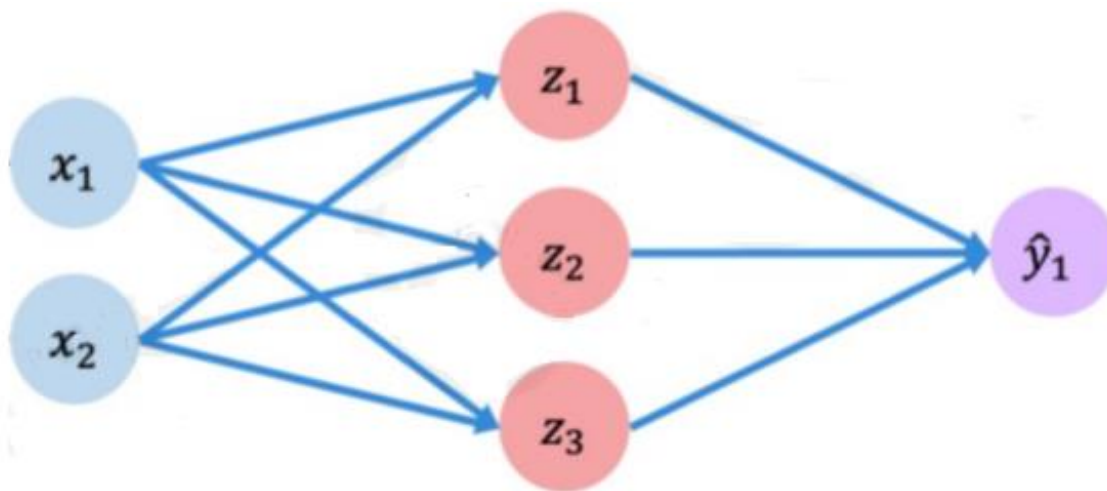
1. 没有软饱和现象。
2. 网络稀疏表达能力, 时间和空间复杂度低。

缺点:

1. 输出值不是Zero-center。
2. 某些神经元可能不被激活

深度学习模型的训练

loss函数的作用就是描述模型的预测值与真实值之间的差距大小。
训练的过程，就是通过不断的迭代计算，使用梯度下降的优化算法，
使得损失函数越来越小。损失函数越小就表示算法达到意义上的最优。



$$\mathcal{L}(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{预测值}}, \underbrace{y^{(i)}}_{\text{真实值}})$$

预测值

真实值

损失函数

交叉熵(Cross Entropy)刻画了概率分布之间的距离，表达预测输入样本属于某一类的概率，更适合用于输出概率的模型，例如分类问题

$$J(W) = -\frac{1}{n} \sum_{i=1}^n y^{(i)} \log(f(x^{(i)}; W)) + (1 - y^{(i)}) \log(1 - f(x^{(i)}; W))$$

均方误差MSE (Mean Squared Loss) 是指参数估计值和参数真实值之差平方的期望值，输出连续的实数，适合于回归问题。

$$J(W) = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - f(x^{(i)}; W))^2$$

梯度下降算法

算法：

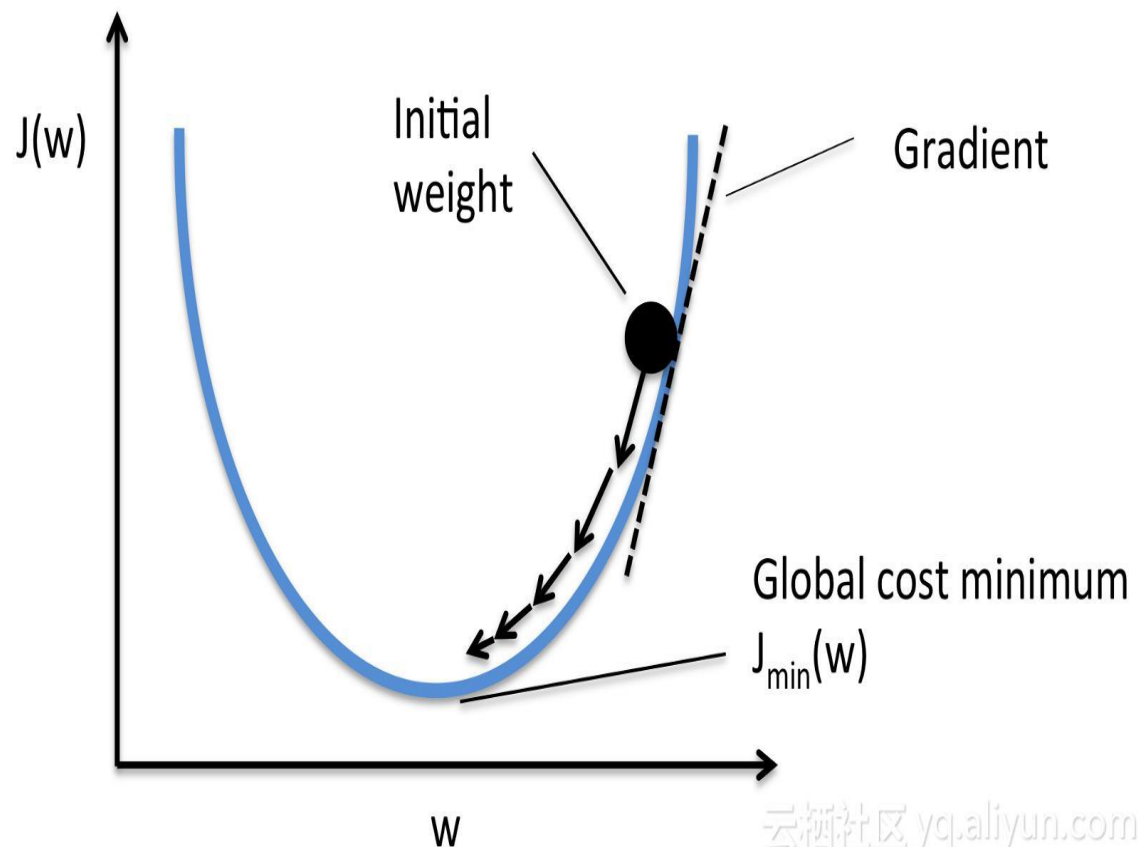
1. 随机初始化权重 w

2. Loop until 收敛：

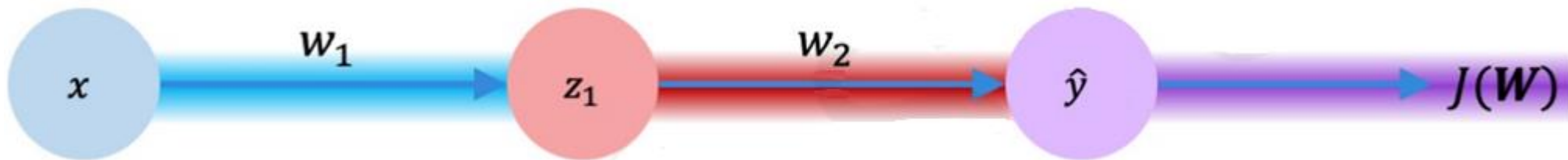
3. 计算梯度, $\frac{\partial J(w)}{\partial w}$

4. 更新权重, $w \leftarrow w - \eta \frac{\partial J(w)}{\partial w}$

5. Return 权重



反向传播 (Backward Propagation)



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

梯度下降算法

算法：

1. 随机初始化权重 W

2. Loop until 收敛：

3. 计算梯度, $\frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(W)}{\partial W}$

4. 更新权重, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$

5. Return 权重

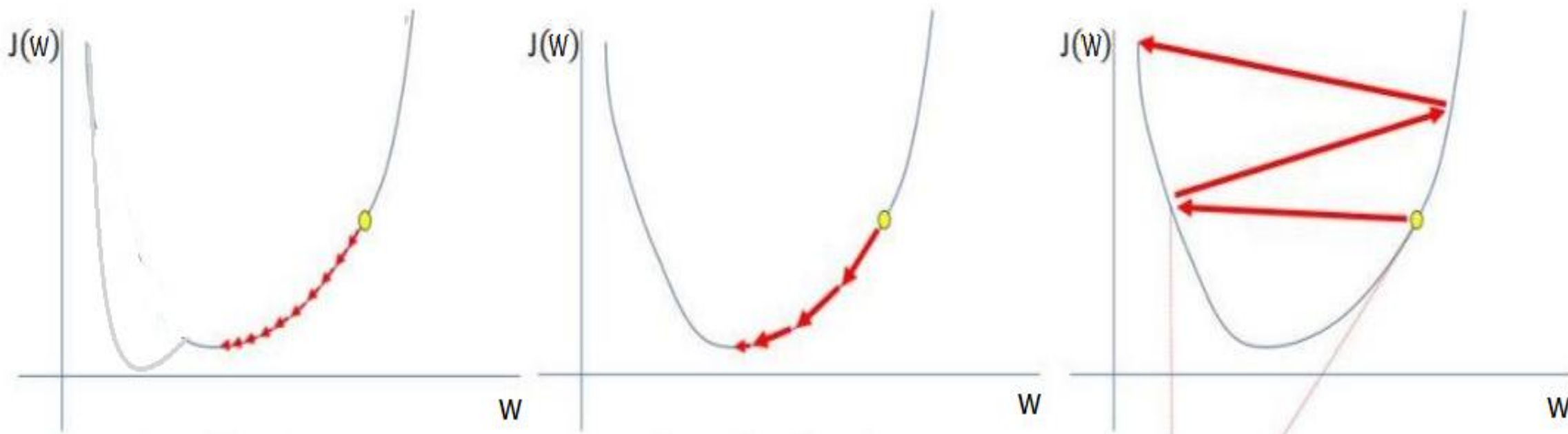
◆ Batch Gradient Descent

◆ Stochastic Gradient Descent

◆ Mini-Batch Gradient Descent

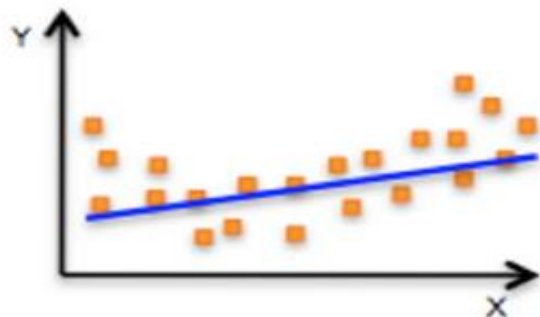
(优点：分布式并行)

学习率 (Learning Rate)

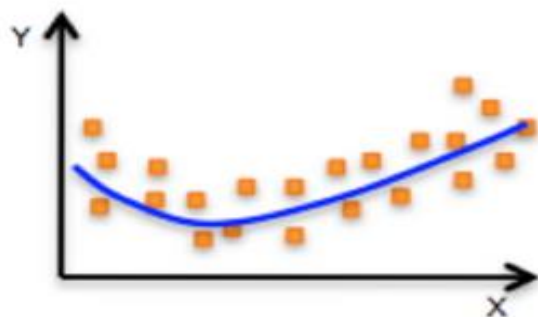


学习率(Learning rate)是深度学习中重要的超参数，决定着模型能否收敛和收敛的速度。

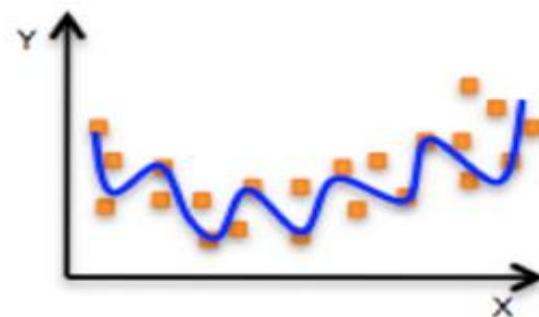
模型训练的问题-过拟合



欠拟合



理想状态



过拟合

在训练集上的表现非常地完美，但是在训练集以外的数据集却表现不好。模型的泛化能力差。

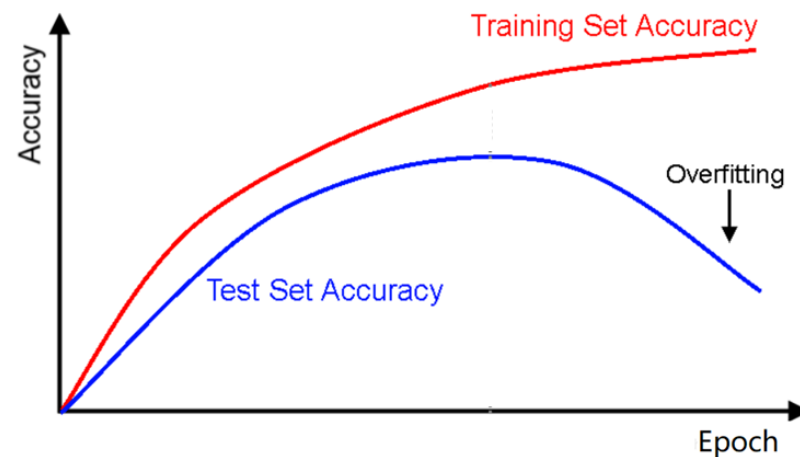
表现：

随着训练周期的增加，训练集的准确率一直上升，但是测试集的准确率不增反降

原因：1.样本：量少、质量低
2.模型：复杂、参数多、不适合

解决方法：

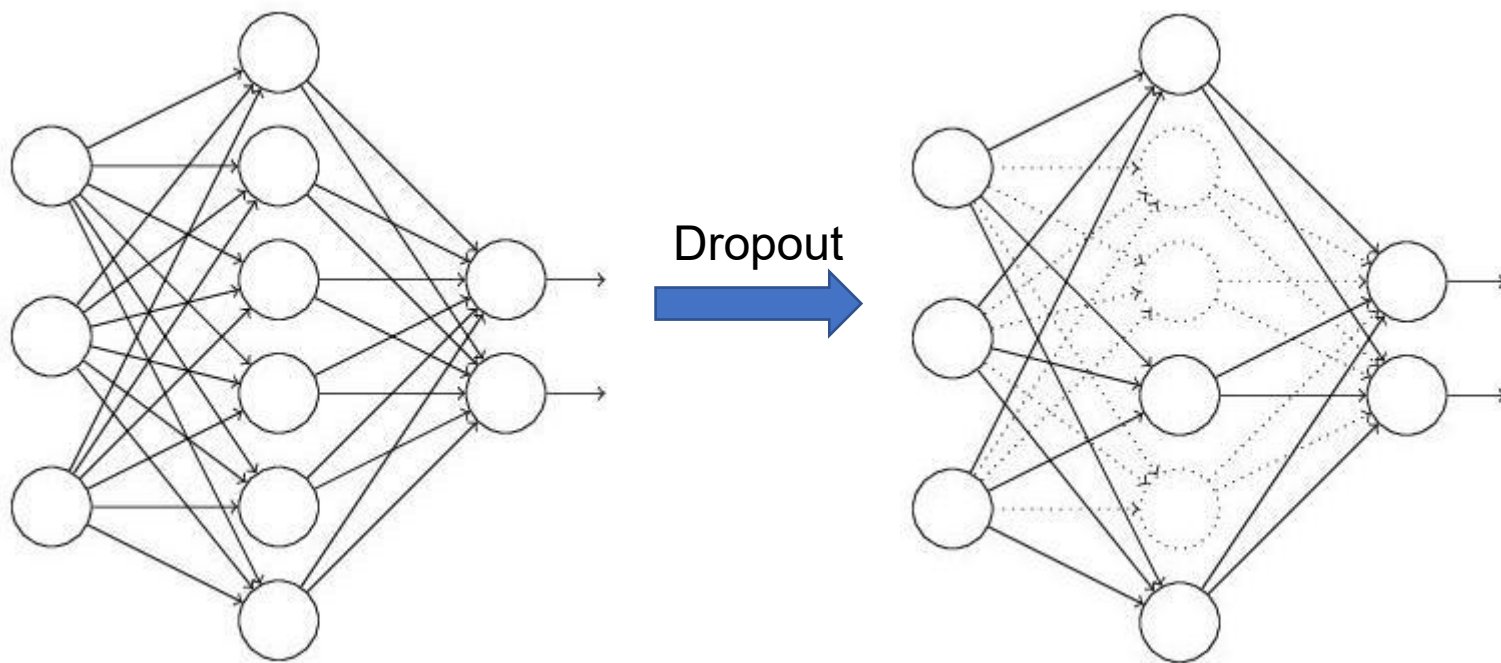
1.样本：数据增强(多样性)，数据筛选(质量)
2.模型：正则化，限制参数大小，减小模型复杂度



过拟合解决方法-Dropout

降低模型复杂度和模型融合：

在每个训练批次中，在一定的概率上（通常设置为0.5）隐式的去除网络中的神经元。



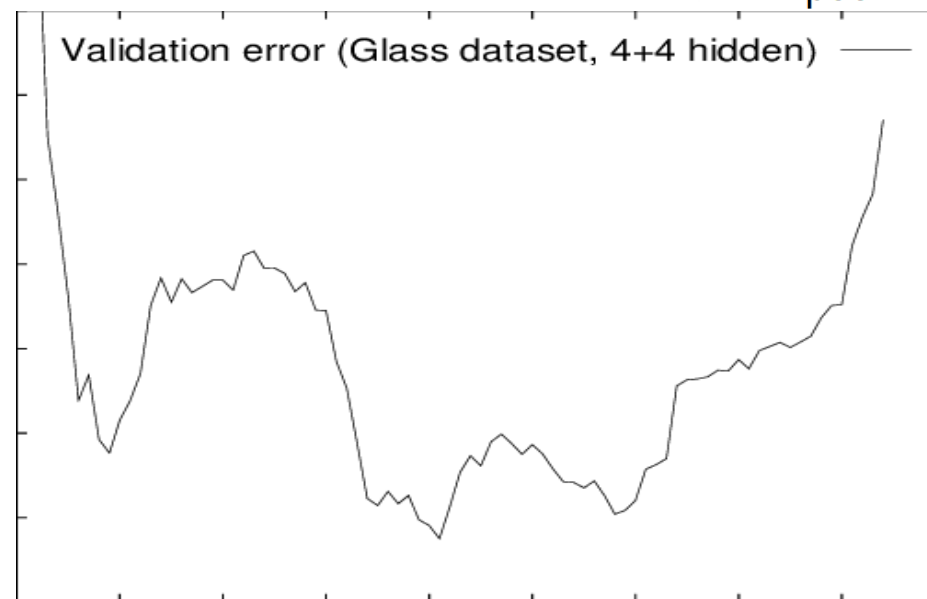
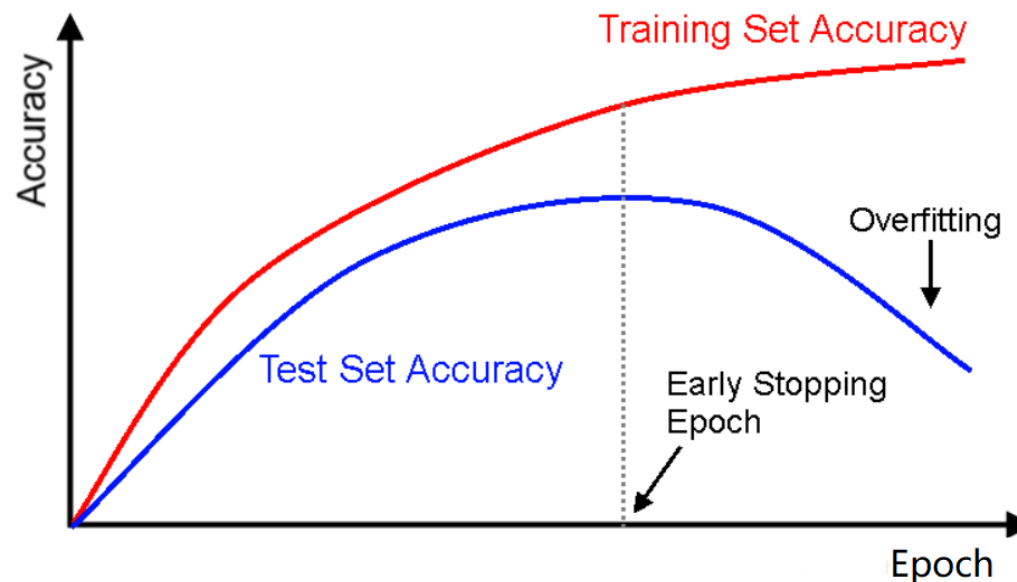
过拟合解决方法-Early Stopping

核心思想：

训练过程中，根据验证集的表现，当验证集的正确率不再提升时就停止训练

停止标准：

- 1)当验证集的错误率比目前最低的验证集错误率超过一定阈值时，停止
- 2)计算当前迭代周期训练集上平均错误率相对于最低错误率的差值，当验证集错误率与这个差值的商大于某个阈值时，停止
- 3) 当验证集错误率在连续 s 个周期内增长的时候，停止

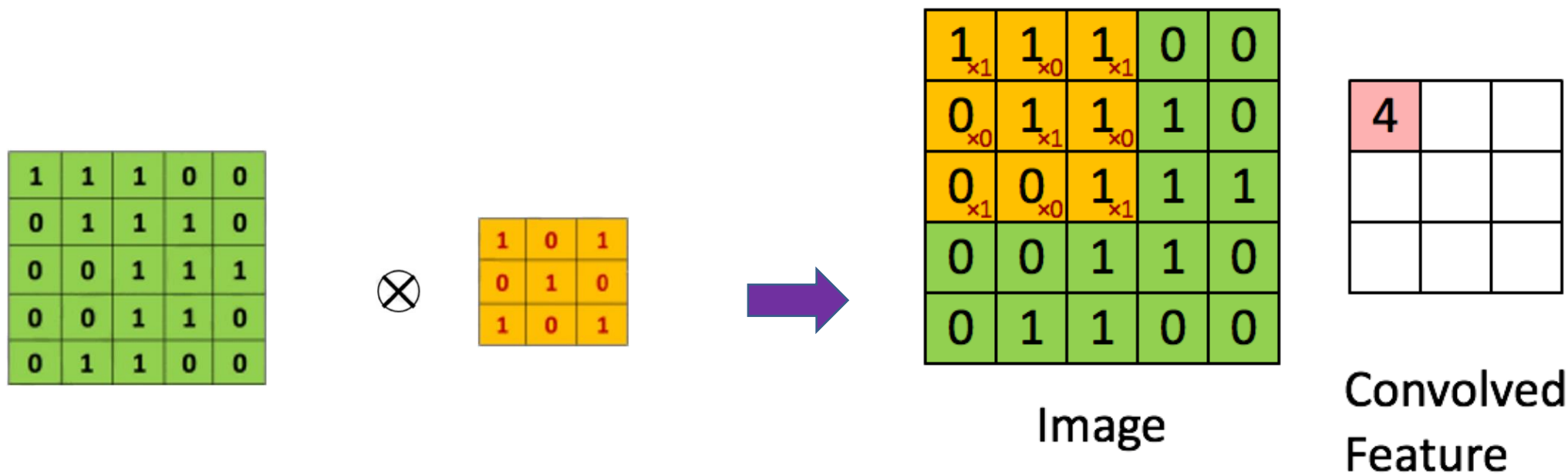
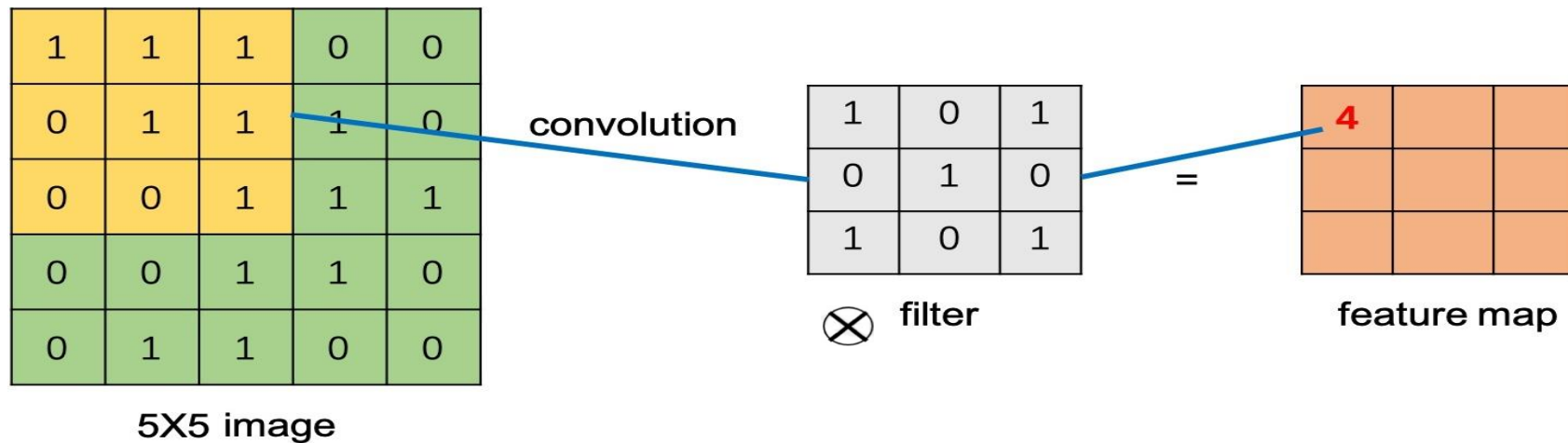




深度学习与机器视觉

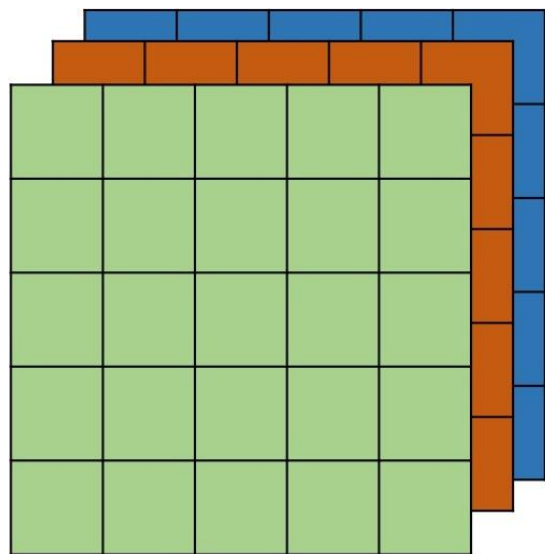
卷积神经网络 (CNN)

卷积计算

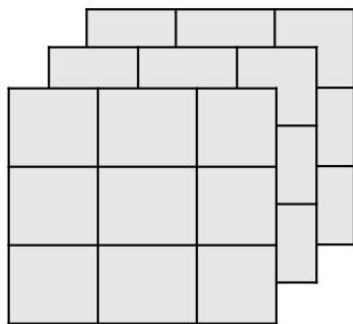


多通道卷积

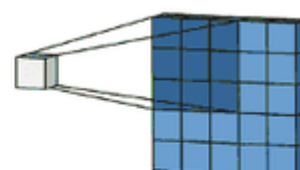
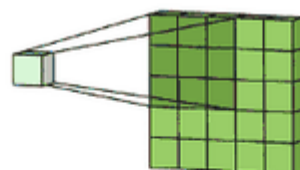
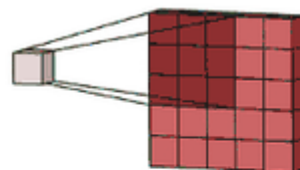
输入是RGB图像，也就是多通道结构。



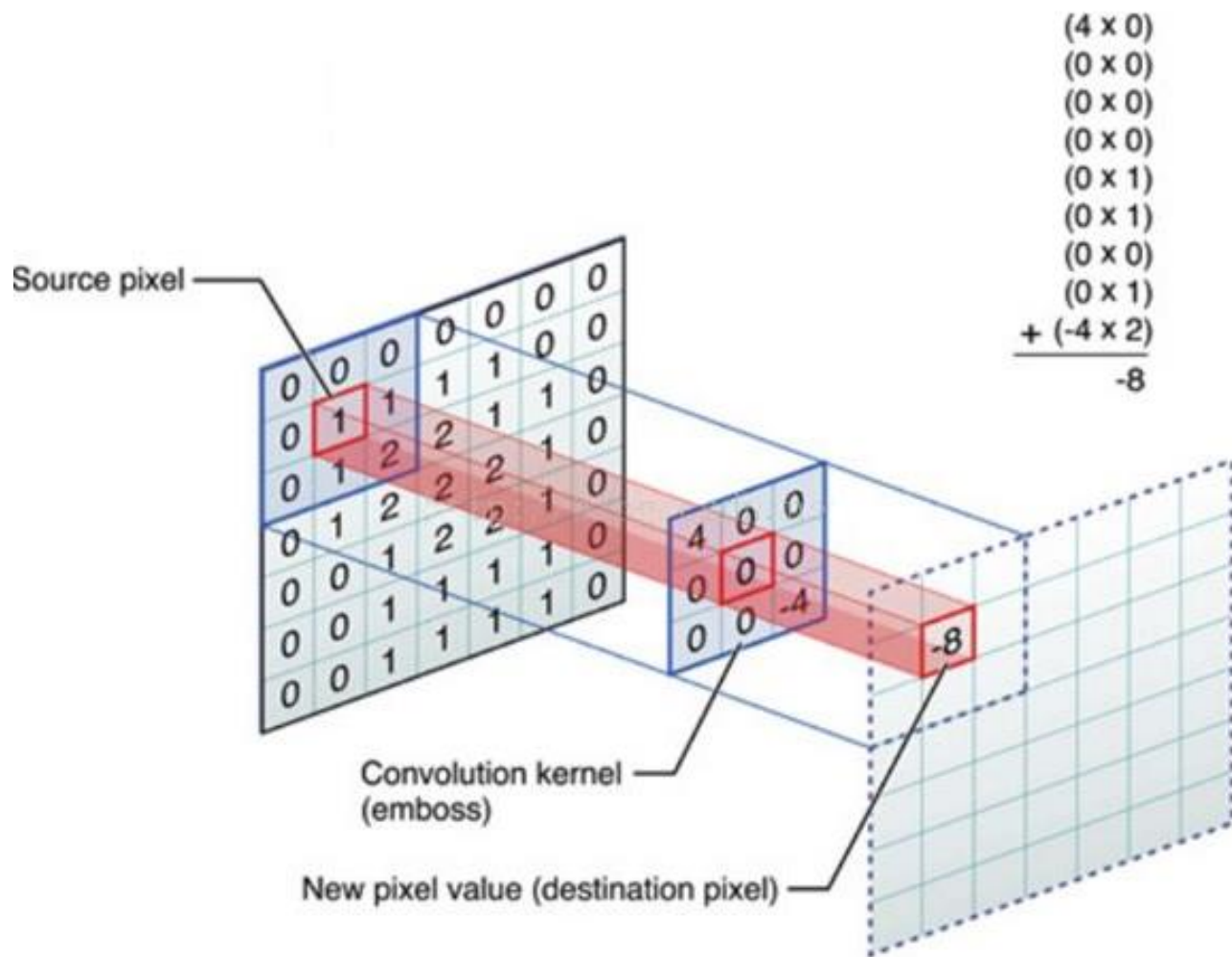
5X5X3 image



3X3X3 filter



卷积



- ✓ 卷积核
- ✓ 原图像和卷积核加权叠加
- ✓ 提取图像局部特征
- ✓ 权值共享

卷积 - Padding

Padding是在输入矩阵数据周围进行数据填充，常用0来填充。

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

5X5 image

0	0	0	0	0	0	0
0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

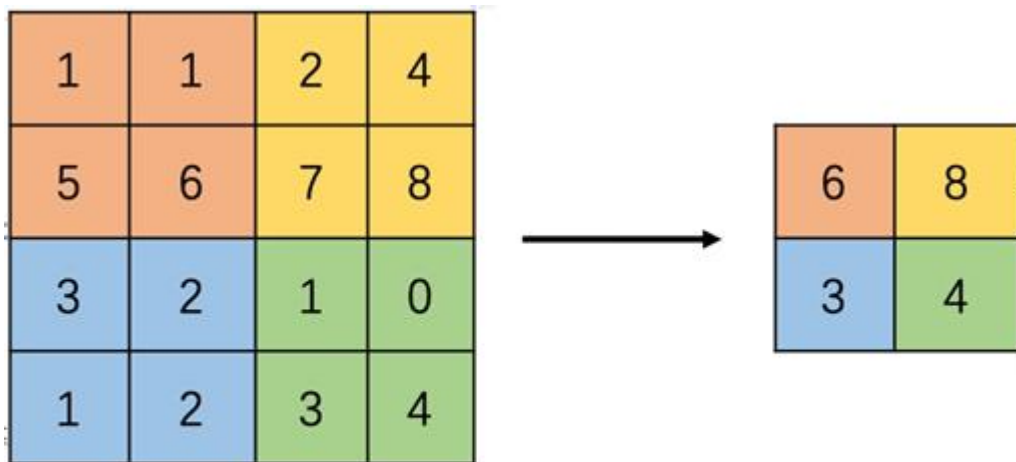
Padding = 1

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0
0	0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

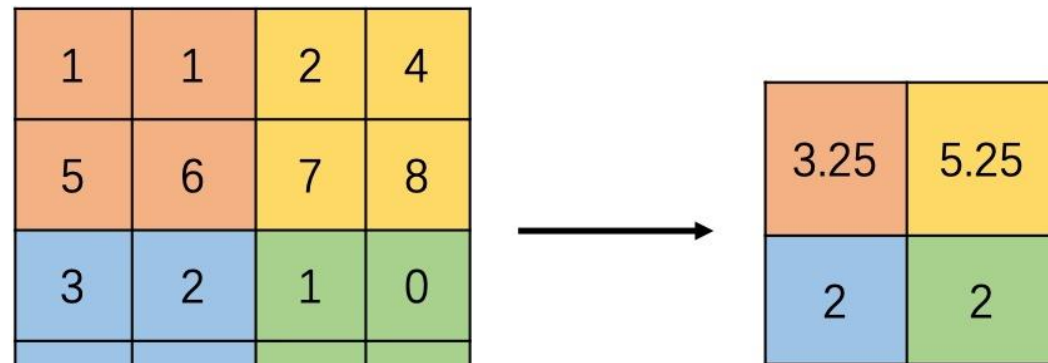
Padding = 2

池化 – Pooling

池化层主要的作用是下采样。



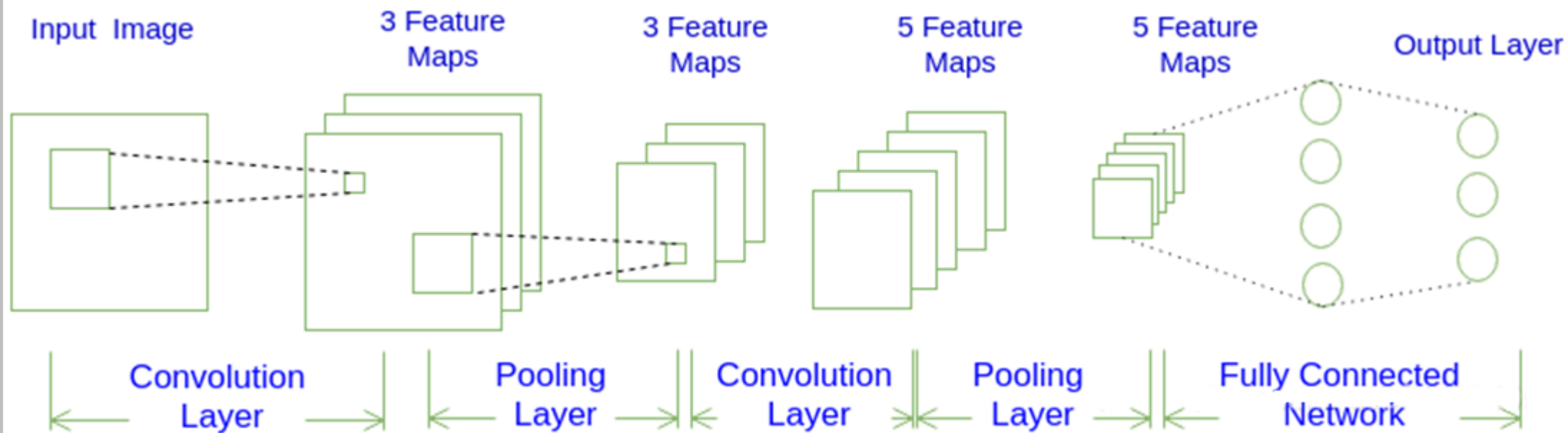
Max-pooling



Average-pooling

卷积神经网络 CNN

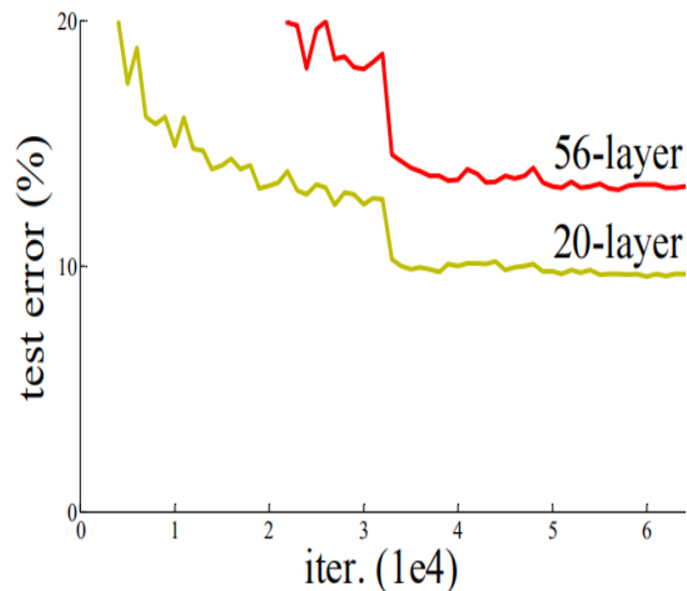
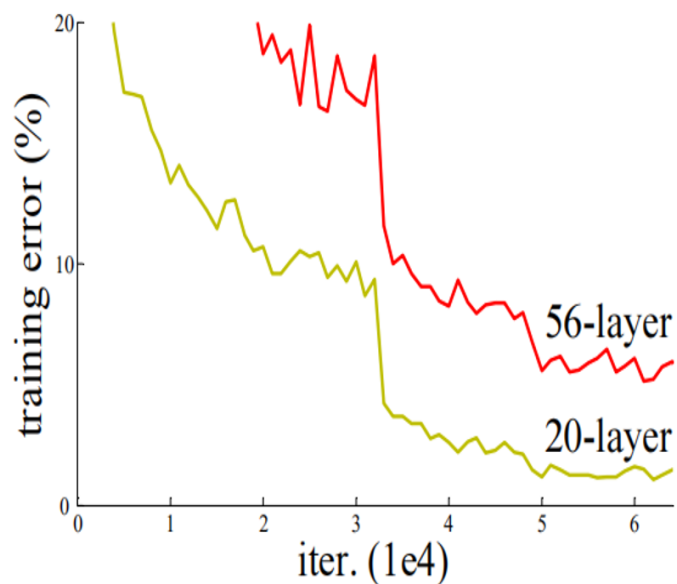
卷积神经网络由若干**卷积层**、**池化层**、**全连接层**组成。
常用结构：INPUT -> [[CONV]*N -> POOL]*M -> [FC]*K



深度网络退化问题

共识：网络的深度对模型的性能至关重要，网络越深、能力越强

退化问题 (Degradation problem)：网络深度增加时，网络准确度出现饱和，甚至出现下降。

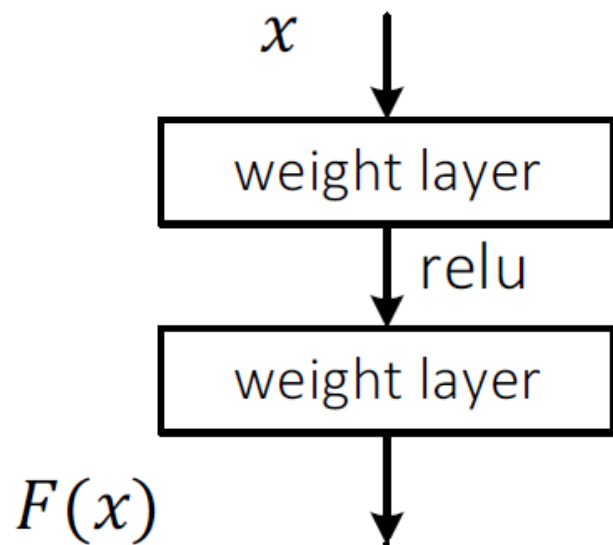


ResNet – 残差网络

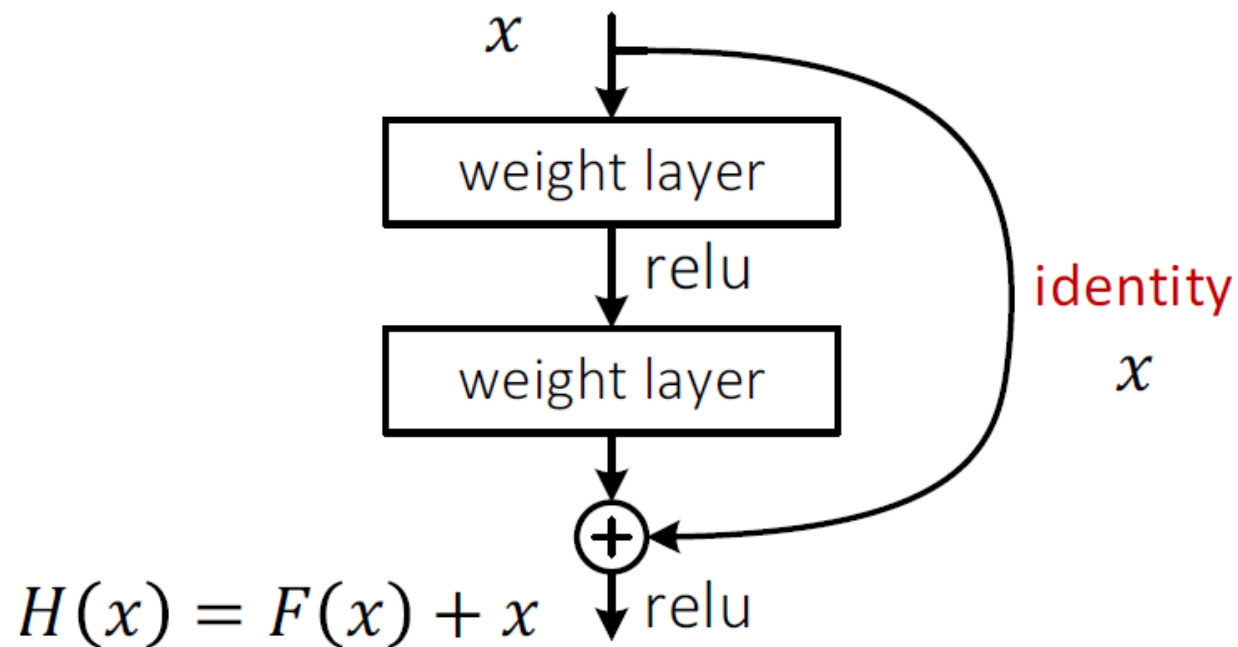
何恺明等《Deep Residual Learning for Image Recognition》，在2016CVPR获得best paper。

残差学习 (Residual learning) - 残差块 (Residual block)

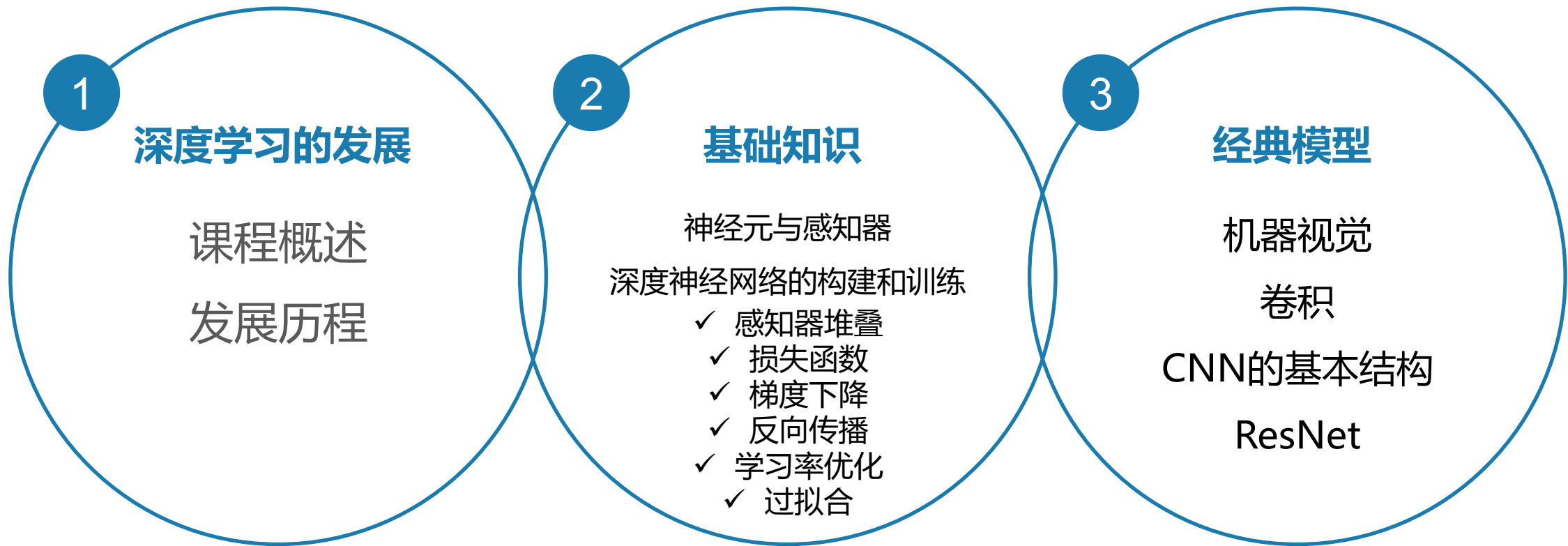
短路连接 (shortcut connection) - 跳跃连接 (skip connection)



- Residual



总结



Q&A